A

Mini Project Report

On

## CROP YIELD & RAINFALL INTELLIGENCE DASHBOARD

# Dynamic Data Visualization Dashboard using Python

*Submitted to DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY*

*In Partial Fulfillment of the requirements for the Award of Degree of*

**BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING(AIML/AI)**

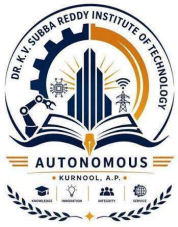
Submitted By

**J SHIVA KRISHNA**
**ROLL NO:-24FH1A3336**

Under the Esteemed guidance of

**Dr. B.MAHESH**

Associate Professor, Department of CSE




**Department of CSE(AI & AIML)**

**DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY**

**UGC AUTONOMOUS**

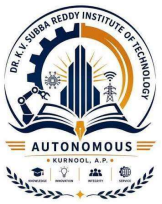
(Approved by AICTE, NEW DELHI, Affiliated to JNTUA, Anantapur)

**2026-2027**

# DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY
# UGC AUTONOMOUS

*(Accredited by NBA,Approved by AICTE NEW DELHI, Affiliated to JNTUA, Anantapur)*

## Department of CSE(AI/AIML)




# CERTIFICATE

This is to certify that the project entitled **"Dynamic Data Visualization Dashboard using Python"** is a bonafide work carried out by the following student

**J SHIVA KRISHNA**
**ROLL NO:-24FH1A3336**

In partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE(AI/AIML)** from Dr.K.V.Subba Reddy Institute of Technology, Kurnool, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

**Guide** **Head of the Department**

# DECLARATION

This is to certify that the work reported in the present Mini project entitled ‘‘Dynamic Data Visualization Dashboard using Python” is a record of bonafide work done by us in the Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology, Kurnool. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future. The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

**J SHIVA KRISHNA**

**ROLL NO:-24FH1A3336**

# ACKNOWLEDGMENT

We are extremely grateful to Dr. K Sree Latha, Principal and H. Ateeq Ahmed, HOD, Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology for their constant support. We are extremely thankful to Dr. B.Mahesh, Associate Professor, Department of CSE, Internal Guide, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

J SHIVA KRISHNA

ROLL NO:-24FH1A3336

# TABLE OF CONTENTS


**1. Abstract** ..... **6**

**2. Introduction** ..... **6**

2.1 Background ..... 6
2.2 Motivation ..... 6
2.3 Problem Statement ..... 6
2.4 Objectives of the Project ..... 6
2.5 Scope of the Project ..... 7

**3. Literature Survey / Existing System** ..... **7**

3.1 Existing Data Visualization Tools ..... 7
3.2 Limitations of Existing System ..... 7
3.3 Proposed System ..... 8

**4. Technologies and Tools Used** ..... **8**

4.1 Python ..... 8
4.2 Streamlit ..... 8
4.3 Pandas ..... 8
4.4 NumPy ..... 8
4.5 Plotly Express ..... 9

**5. System Requirements** ..... **9**

5.1 Hardware Requirements ..... 9
5.2 Software Requirements ..... 9

**6. Dataset Description** ..... **10**

6.1 Dataset Sources ..... 10
6.2 Dataset Attributes ..... 10
6.3 Data Types ..... 10
6.4 Data Preprocessing ..... 10
6.5 Data Cleaning ..... 11

**7. Implementation** ..... **11**

7.1 Project Structure ..... 11
7.2 Data Loading ..... 11
7.3 Data Preprocessing ..... 11
7.4 Visualization Development ..... 11
7.5 Streamlit Interface ..... 11
7.6 Interactive Filters ..... 11
7.7 Dashboard Integration ..... 12

**8. Data Visualization** ..... **12**

8.1 Production Trend Chart ..... 12
8.2 Crop Production Share and Crop Yield Charts ..... 13
8.3 Rainfall and Weather Visualization ..... 13
8.4 Regional Comparison and Correlation Heatmap ..... 13

**9. Results and Discussion** ..... **13**

9.1 Dashboard Screenshots ..... 13
9.2 Visualization Results ..... 14
9.3 Analysis of Results ..... 15
9.4 Key Insights ..... 15

**10. Advantages and Limitations** ..... **15**

10.1 Advantages ..... 15
10.2 Limitations ..... 15

**11. Conclusion** ..... **16**

**12. Future Enhancement** ..... **16**

**13. References** ..... **16**

**14. Appendix** ..... **17**

A. Source Code ..... 17
B. Dataset Notes ..... 19
C. Additional Screenshots ..... 20

# 1. Abstract

The Crop Yield & Rainfall Intelligence Dashboard is an interactive data-visualization mini project developed with Python, Streamlit, Pandas, and Plotly. It helps users explore agricultural production, crop yield, cultivated area, rainfall, temperature, humidity, season, and region. Users can filter records by year range, state, crop, and season; compare crops and states; inspect trends and relationships; view filtered records; and download filtered data as CSV.

The dashboard organises its output into five tabs (Overview, Crop Analysis, Rainfall, Regional Comparison, and Data & Insights) and four key performance indicators that update with every filter change. The included dataset contains 600 simulated records covering 2015–2024 and is intended for educational demonstration. It is not official agricultural statistics.

**Keywords:** Data Visualization, Streamlit, Pandas, Plotly, Crop Yield, Rainfall, Exploratory Data Analysis, Dashboard.

# 2. Introduction

## 2.1 Background

Agriculture is influenced by connected factors such as rainfall, temperature, humidity, cultivated area, crop type, and growing season. Agricultural records are usually stored as large tables with one row per year, region, and crop. Raw tables of this kind make it difficult to notice changes over time or to compare regions and crops.

Data visualization converts records into charts and summaries that are easier to inspect. Interactive dashboards go a step further: the viewer can change the question being asked (a different state, crop, or time window) and immediately see an updated answer without writing any code.

## 2.2 Motivation

Spreadsheets and static charts answer only one fixed question. Students and analysts who want to explore agricultural data need a tool that is quick to build, simple to run, and flexible enough to be pointed at a different CSV file. Python's data ecosystem makes this possible with a small amount of code, which motivated the development of this dashboard as a hands-on exercise in exploratory data analysis and interactive visualization.

## 2.3 Problem Statement

Crop production, yield, and weather records are difficult to interpret in tabular form. There is a need for a simple web-based dashboard that allows users to filter the data by year, state, crop, and season, view summary indicators, compare crops and regions, examine rainfall–yield relationships, and export the filtered records, all from a single interface.

## 2.4 Objectives of the Project

- Visualize year-wise agricultural production trends.
- Compare average yield, production, and cultivated area across crop types.
- Explore rainfall, temperature, humidity, and crop-yield patterns.

- Compare state-level yield and production summaries.
- Provide interactive filters for year, state, crop, and season.
- Show correlations among numeric variables.
- Inspect and download filtered records.
- Support analysis of another CSV with the same column structure.

### 2.5 Scope of the Project

The project covers loading a CSV file, cleaning it, filtering it through a sidebar, computing summary statistics, and presenting the results as interactive Plotly charts inside a Streamlit application. It is a learning tool for visualization and exploratory data analysis. It does not build predictive models and does not establish that rainfall alone causes changes in yield. The supplied values are simulated rather than verified agricultural measurements; real-world analysis would require reliable data and consideration of irrigation, soil, seed variety, pests, fertilizer, and other factors.

## 3. Literature Survey / Existing System

### 3.1 Existing Data Visualization Tools

Several tools are commonly used to visualize tabular data:

- **Spreadsheet software (Microsoft Excel, Google Sheets):** easy to use and widely available, offering pivot tables and basic charts.
- **Business-intelligence tools (Tableau, Power BI):** powerful interactive dashboards with drag-and-drop design, but often requiring licences and a separate learning curve.
- **Static plotting libraries (Matplotlib, Seaborn):** produce publication-quality images from Python, but the output is not interactive by default.
- **Government and open-data portals:** publish agricultural and weather statistics, typically as downloadable tables or fixed reports.

### 3.2 Limitations of Existing System

- Spreadsheet charts must be rebuilt manually when the question changes, and large multi-dimensional filtering becomes cumbersome.
- Commercial BI tools can be costly and are heavier than needed for a small academic project.
- Static charts do not allow hovering, zooming, or filtering, so users cannot explore the data themselves.
- Published reports present fixed views; combining rainfall, yield, area, and region in one interactive place usually requires extra effort.

### 3.3 Proposed System

The proposed system is a Python-based dashboard built with Streamlit, Pandas, and Plotly. It loads a CSV file, applies sidebar filters, recalculates indicators, and redraws interactive charts on every change. Because it is written entirely in Python and runs from a single script, it is free, easy to modify, and can analyse any CSV file that follows the same column structure through the built-in upload option.

## 4. Technologies and Tools Used

The dashboard is assembled from a small set of open-source Python libraries.

| Technology | Purpose |
|---|---|
| Python | Application programming language |
| Streamlit | Dashboard interface, filters, metrics, upload and download |
| Pandas | CSV loading, cleaning, filtering, grouping and calculations |
| Plotly Express | Interactive line, bar, pie, scatter and heatmap charts |
| CSV | Tabular dataset format |
| requirements.txt | Lists required Python packages |

### 4.1 Python

Python is a general-purpose, high-level programming language with a large ecosystem for data analysis. The entire dashboard, from data loading to chart creation, is written in a single Python script. The pathlib module is used to locate the dataset file next to the script.

### 4.2 Streamlit

Streamlit is an open-source framework that turns Python scripts into interactive web applications. In this project it provides the page configuration, sidebar, file uploader, multiselect and slider filters, metric cards, tabs, columns, data tables, and the CSV download button. The decorator `@st.cache_data` is used so that the dataset is not re-read from disk on every interaction.

### 4.3 Pandas

Pandas provides the DataFrame structure used to hold the records. It is used to read the CSV file, convert columns to numeric types, drop incomplete rows, filter by conditions, group records by year, crop, or state with `groupby` and `agg`, and compute the correlation matrix.

### 4.4 NumPy

NumPy is not imported directly in the dashboard script. It is, however, installed automatically as a dependency of Pandas and Plotly and performs the numerical array operations that Pandas relies on internally.

### 4.5 Plotly Express

Plotly Express is the visualization library actually used for all charts. It creates interactive figures (hover tooltips, zoom, pan, legend toggling, and image export) with one function call each. The functions used are `px.line`, `px.pie`, `px.scatter`, `px.bar`, and `px.imshow`. The OLS trendline in the Rainfall tab additionally requires the `statsmodels` package.

## 5. System Requirements

### 5.1 Hardware Requirements

The dashboard is lightweight, since the bundled dataset is only 40.4 KB. The following configuration is sufficient:

| Component | Minimum requirement |
|---|---|
| Processor | Dual-core processor, 2 GHz or higher |
| RAM | 4 GB (8 GB recommended) |
| Storage | At least 1 GB free disk space for Python and libraries |
| Display | 1366 x 768 resolution or higher (wide screen recommended for the wide layout) |
| Network | Not required to run locally; needed only to install packages |

### 5.2 Software Requirements

| Software | Requirement |
|---|---|
| Operating system | Windows, macOS, or Linux |
| Python | Version 3.10 or newer |
| Python packages | streamlit, pandas, plotly (statsmodels is needed for the OLS trendline in the Rainfall tab) |
| Web browser | Any modern browser (Chrome, Edge, Firefox) |
| Code editor | VS Code, PyCharm, or any text editor |

**Installation and execution**

- Install Python 3.10 or newer.
- Keep crop_yield_dashboard.py and crop_yield_rainfall_dataset.csv in the same folder.
- Open a terminal in the project folder.
- Install dependencies: `pip install streamlit pandas plotly statsmodels`
- Run: `streamlit run crop_yield_dashboard.py`
- Open the local URL shown in the terminal, usually http://localhost:8501.

# 6. Dataset Description

## 6.1 Dataset Sources

The CSV file `crop_yield_rainfall_dataset.csv` bundled with the project is a **simulated educational dataset** created for demonstration. It was not downloaded from any official source. It contains 600 rows and 10 columns, covering 2015–2024, 10 state/region labels, 6 crop types, and 3 seasons. Each row represents a simulated year–state–crop–season record. File size: 40.4 KB. Missing cells: 0.

**Important dataset note:** The CSV is simulated educational data, not official or verified agricultural data. Do not use it for agricultural, commercial, scientific, or policy decisions.

## 6.2 Dataset Attributes

| Column | Meaning / unit |
|---|---|
| Year | Calendar year |
| State | State or region label |
| Crop | Crop category |
| Season | Kharif, Rabi, or Summer |
| Rainfall_mm | Simulated rainfall in millimetres |
| Temperature_C | Simulated temperature in degrees Celsius |
| Humidity_pct | Simulated relative humidity percentage |
| Area_Hectares | Simulated cultivated area in hectares |
| Yield_Tonnes_per_Hectare | Simulated yield in tonnes per hectare |
| Production_Tonnes | Simulated production in tonnes |

## 6.3 Data Types

| Type | Columns |
|---|---|
| Numeric (converted with pd.to_numeric) | Year, Rainfall_mm, Temperature_C, Humidity_pct, Area_Hectares, Yield_Tonnes_per_Hectare, Production_Tonnes |
| Categorical (text) | State, Crop, Season |

## 6.4 Data Preprocessing

After the CSV is read, the numeric columns listed above are converted using `pd.to_numeric(..., errors="coerce")`. Values that cannot be converted become missing values (NaN) instead of causing an error. Year values are later converted to integers to build the year-range slider, and the categorical columns are sorted to build the filter options.

### 6.5 Data Cleaning

Rows missing any of the essential fields Year, State, or Crop are removed with `dropna(subset=["Year", "State", "Crop"])`, because these fields are needed for filtering and grouping. The bundled dataset has no missing cells, but this step makes the dashboard robust when a different CSV file is uploaded. If the dataset file is not found and no file is uploaded, the application shows an error message and stops instead of crashing.

## 7. Implementation

### 7.1 Project Structure

| File | Description |
|---|---|
| crop_yield_dashboard.py | Main Streamlit application containing all logic and charts |
| crop_yield_rainfall_dataset.csv | Simulated dataset (600 rows, 10 columns) |
| requirements.txt | List of required Python packages |

### 7.2 Data Loading

The function `load_data()` is decorated with `@st.cache_data`. It reads the uploaded CSV if the user supplied one, otherwise it reads the bundled file located beside the script, and otherwise returns an empty DataFrame. Caching prevents the file from being re-read each time a filter changes.

### 7.3 Data Preprocessing

Inside the same function, the seven numeric columns are converted to numeric types and rows with missing Year, State, or Crop are dropped. After loading, the sidebar code builds the list of available years, states, crops, and seasons from the cleaned data.

### 7.4 Visualization Development

Every chart is created with Plotly Express from either the filtered DataFrame or a grouped summary of it. Typical steps are: (1) group the filtered records with `groupby().agg()`, (2) pass the result to a `px` function with titles and axis labels, and (3) render it with `st.plotly_chart(fig, use_container_width=True)` so that it fills its column.

### 7.5 Streamlit Interface

The page uses a wide layout with an expanded sidebar. The main area shows the title, a caption, an information banner about the simulated data, four KPI cards (total production, average yield, average rainfall, total cultivated area), and five tabs. Two-column layouts (`st.columns`) place related charts side by side.

### 7.6 Interactive Filters

The sidebar contains the controls listed below. The dataset is filtered with a single boolean expression that combines all conditions, and a warning is shown if no rows match.

| Control | Streamlit widget | Filters |
|---|---|---|
| Replacement CSV | file_uploader | Replaces the bundled dataset |
| Year range | select_slider | Year (start and end) |
| State / region | multiselect | State |
| Crop | multiselect | Crop |
| Season | multiselect | Season |

### 7.7 Dashboard Integration

All components are integrated in one top-to-bottom script: the page configuration, data loading, sidebar filters, filtered DataFrame, KPI calculation, and the five tabs. Streamlit re-runs the script whenever a filter changes, so the KPI cards, charts, tables, and quick insights always agree with the current selection. The complete source code is provided in Appendix A.

## 8. Data Visualization

The dashboard contains the following visualizations, grouped by tab.

| Tab | Chart | Type (Plotly function) |
|---|---|---|
| Overview | Total production by year | Line chart with markers (px.line) |
| Overview | Production share by crop | Donut chart (px.pie, hole=0.35) |
| Overview | Rainfall vs yield | Bubble scatter plot (px.scatter) |
| Crop Analysis | Average yield by crop; total production by crop | Vertical and horizontal bar charts (px.bar) |
| Rainfall | Average rainfall and temperature trend | Multi-line chart (px.line) |
| Rainfall | Rainfall–yield relationship | Scatter plot with OLS trendline (px.scatter) |
| Regional Comparison | Top 10 states by average yield and by production | Bar charts (px.bar) |
| Data & Insights | Correlation heatmap | Heatmap (px.imshow) |

### 8.1 Production Trend Chart

The line chart on the Overview tab sums *Production_Tonnes* for each year and plots it with markers. It shows how total output rises and falls over the selected period and responds to every filter, so the user can follow the trend of a single crop or state. Hovering over a marker displays the exact value.

### 8.2 Crop Production Share and Crop Yield Charts

The donut chart shows each crop's share of total production, while the Crop Analysis tab compares average yield per crop and total production per crop. Because crops are measured on very different scales, these charts make it clear which crops dominate production and which have the highest yield per hectare. A summary table beneath the charts lists average yield, production, and area for each crop.

### 8.3 Rainfall and Weather Visualization

The Rainfall tab plots the yearly average rainfall and temperature as lines and then shows a scatter plot of rainfall against yield, coloured by season, with an ordinary-least-squares trendline. The Overview tab also contains a bubble chart of the same two variables in which bubble size represents cultivated area. A caption warns that a visible relationship is not proof that rainfall alone causes changes in yield.

### 8.4 Regional Comparison and Correlation Heatmap

The Regional Comparison tab ranks the top 10 states by average yield and by total production and lists average yield, production, and average rainfall for each state. The Data & Insights tab presents a correlation heatmap for Rainfall, Temperature, Humidity, Area, Yield, and Production using a red–yellow–green colour scale from −1 to +1, followed by the filtered records, a CSV download button, and automatically generated quick insights.

## 9. Results and Discussion

### 9.1 Dashboard Screenshots

The following screenshots were captured from the running dashboard. The charts depend on the filter selections visible in the screenshots.

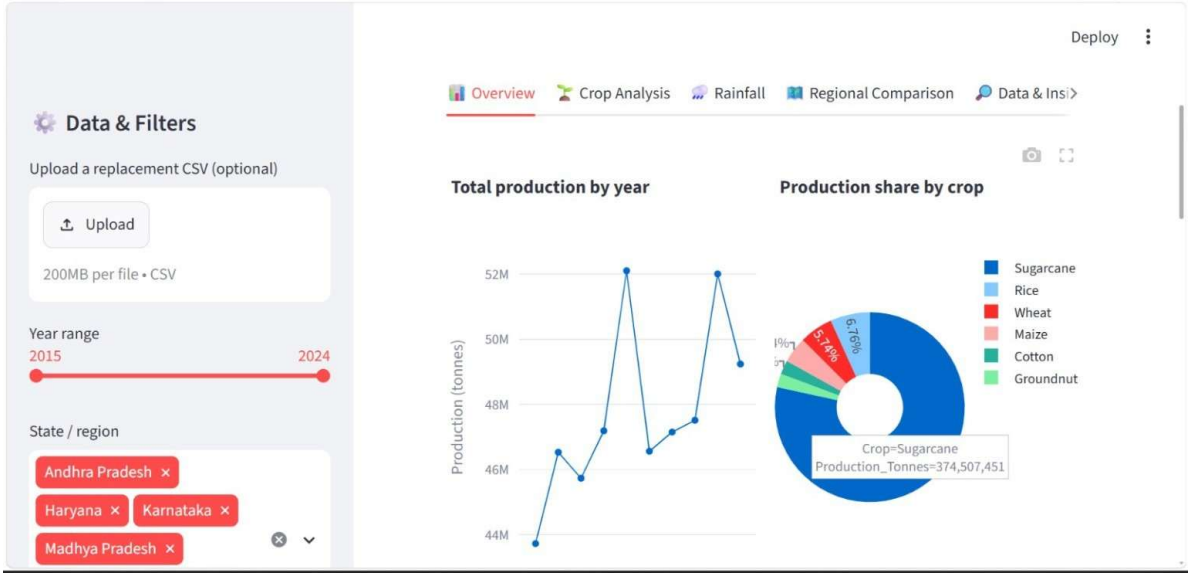



*Figure 1. Overview tab: total production by year and production share by crop.*




*Figure 2. Crop Analysis tab: average yield and total production by crop.*

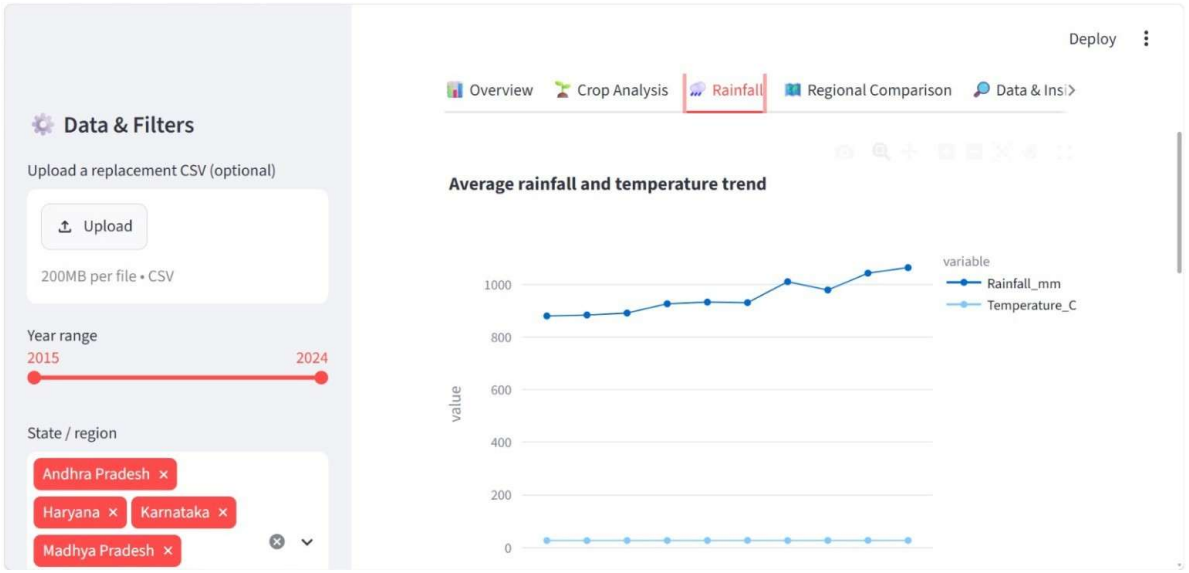



*Figure 3. Rainfall tab: average rainfall and temperature trend.*

## 9.2 Visualization Results

**Overview tab.** The production line (Figure 1) fluctuates from year to year rather than rising steadily, and the donut chart shows that sugarcane accounts for the overwhelming majority of total production, with rice, wheat, maize, cotton, and groundnut making up small slices.

**Crop Analysis tab.** Figure 2 shows that sugarcane has a far higher average yield and total production than every other crop, which compresses the other bars in the same chart.

**Rainfall tab.** Figure 3 shows average rainfall increasing gradually across the years, from roughly 880 mm to a little above 1,000 mm. The temperature line appears almost flat at the bottom of the chart because it is drawn on the same axis as rainfall.

### 9.3 Analysis of Results

Dataset-level summary: total production across the simulated records is 477,738,695 tonnes, mean yield is 13.29 tonnes/hectare, mean rainfall is 954.2 mm, and the summed cultivated area is 41,171,910 hectares. These are calculations from the supplied simulated CSV, not official findings, and the dashboard metrics change when filters are applied.

| Crop | Average yield (t/ha, simulated sample) |
|---|---|
| Sugarcane | 65.83 |
| Rice | 3.72 |
| Wheat | 3.27 |
| Maize | 2.99 |
| Groundnut | 2.08 |
| Cotton | 1.86 |

The mean yield of 13.29 t/ha is much higher than the yield of five of the six crops because sugarcane, a high-biomass crop, is included in the average. For this reason crop-wise or filtered views are more meaningful than the all-crop average.

### 9.4 Key Insights

- Sugarcane has the highest average yield (65.83 t/ha) and the largest share of production in the sample.
- Cotton has the lowest average yield (1.86 t/ha) and groundnut is next lowest (2.08 t/ha).
- Average rainfall shows an upward trend over 2015–2024 in the selected view (Figure 3).
- Production varies from year to year, so single-year comparisons should be made carefully.
- All results describe the simulated sample only and must not be treated as real agricultural findings.

## 10. Advantages and Limitations

### 10.1 Advantages

- Interactive: filters update every chart, KPI, and table instantly.
- Built entirely with free, open-source Python libraries.
- Simple to run: one script, one dataset, and a single command.
- Works with any CSV that follows the same column structure through the upload option.
- Allows filtered data to be downloaded as a CSV file for further analysis.
- Combines multiple views (trend, crop, rainfall, region, correlation) in one interface.
- Reminds users that the data is simulated and that correlation does not imply causation.

### 10.2 Limitations

- The dataset is simulated and is not suitable for real agricultural conclusions.

- Rainfall–yield patterns do not prove causation; irrigation, soil, seed variety, pests, fertilizer, and other factors may affect yield.
- The current dataset uses selected state/region labels and crop categories rather than verified district-level coverage.
- Sugarcane's very high yield compresses the other crops in the yield bar chart, and temperature is hard to read when plotted on the same axis as rainfall.
- The OLS trendline needs the extra statsmodels package, which is not part of the basic install command.
- The dashboard is descriptive only; it does not forecast yield.

## 11. Conclusion

The dashboard demonstrates how an interactive interface can make a tabular dataset easier to explore. Streamlit provides the interface, Pandas supports preparation and analysis, and Plotly creates interactive visualizations. The project includes filters, comparison charts, a correlation heatmap, quick summaries, and filtered-data export.

Replacing the simulated dataset with verified data and documenting its source and units would be the next step toward a practical agricultural analysis tool.

## 12. Future Enhancement

- Replace the sample with official, documented datasets and record their sources and units.
- Add district-level maps for regional comparison.
- Add crop-specific forecasting evaluated against real observations.
- Plot temperature on a secondary axis and offer a log scale or per-crop selector for yield comparisons.
- Include statsmodels in requirements.txt so that the trendline works on a fresh installation.
- Add soil, irrigation, fertilizer, and market-price attributes for deeper analysis.
- Deploy the dashboard online so that others can use it without installing Python.

## 13. References


- Project source code and files: https://github.com/shivakrishna3336-debug/Data-Visualization-Project/tree/main
- Streamlit documentation: https://docs.streamlit.io
- Pandas documentation: https://pandas.pydata.org/docs
- Plotly Express documentation: https://plotly.com/python/plotly-express
- Python documentation: https://docs.python.org/3
- Packages listed in requirements.txt: streamlit, pandas, plotly.

- Potential sources for a future verified-data version: India's Directorate of Economics and Statistics, data.gov.in, the India Meteorological Department (IMD), and FAOSTAT. These are suggestions only; the included CSV was not downloaded from them.

# 14. Appendix

## A. Source Code

The complete source code of `crop_yield_dashboard.py` is given below (202 lines). Emoji icons used in the page title, tab names, and sidebar header are omitted from this printed listing because the PDF font cannot display them; they are present in the actual .py file.

```
"""
Crop Yield & Rainfall Intelligence Dashboard
--------------------------------------------
Run:
    pip install streamlit pandas plotly
    streamlit run crop_yield_dashboard.py

IMPORTANT DATA NOTE:
The included CSV is a simulated educational dataset created for demonstration.
It is not official agricultural statistics and should not be used for policy,
commercial decisions, or scientific conclusions. Replace it with a verified
public dataset for real-world analysis.
"""
from pathlib import Path
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(
    page_title="Crop Yield & Rainfall Intelligence",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded",
)

DATA_FILE = Path(__file__).with_name("crop_yield_rainfall_dataset.csv")

@st.cache_data
def load_data(uploaded_file=None):
    if uploaded_file is not None:
        data = pd.read_csv(uploaded_file)
    elif DATA_FILE.exists():
        data = pd.read_csv(DATA_FILE)
    else:
        return pd.DataFrame()
    numeric_cols = [
        "Year", "Rainfall_mm", "Temperature_C", "Humidity_pct",
        "Area_Hectares", "Yield_Tonnes_per_Hectare", "Production_Tonnes"
    ]
    for col in numeric_cols:
        if col in data.columns:
            data[col] = pd.to_numeric(data[col], errors="coerce")
    return data.dropna(subset=["Year", "State", "Crop"])

st.title("Crop Yield & Rainfall Intelligence Dashboard")
st.caption("Explore crop production, weather patterns, and yield across years and regions.")
st.info(
    "Educational demo: the bundled dataset is simulated for learning and dashboard testing. "
    "Use the CSV upload option to analyse verified public data."
)

with st.sidebar:
    st.header("Data & Filters")
    uploaded = st.file_uploader("Upload a replacement CSV (optional)", type=["csv"])
    data = load_data(uploaded)
    if data.empty:
        st.error("Dataset not found. Keep crop_yield_rainfall_dataset.csv beside this Python file.")
        st.stop()

    years = sorted(data["Year"].dropna().astype(int).unique().tolist())
    year_range = st.select_slider(
        "Year range",
```

```
        options=years,
        value=(years[0], years[-1]),
    )
    state_options = sorted(data["State"].dropna().unique().tolist())
    selected_states = st.multiselect("State / region", state_options, default=state_options)
    crop_options = sorted(data["Crop"].dropna().unique().tolist())
    selected_crops = st.multiselect("Crop", crop_options, default=crop_options)
    season_options = sorted(data["Season"].dropna().unique().tolist())
    selected_seasons = st.multiselect("Season", season_options, default=season_options)

filtered = data[
    data["Year"].between(year_range[0], year_range[1])
    & data["State"].isin(selected_states)
    & data["Crop"].isin(selected_crops)
    & data["Season"].isin(selected_seasons)
].copy()

if filtered.empty:
    st.warning("No rows match these filters. Select at least one state, crop, and season.")
    st.stop()

# KPIs
total_production = filtered["Production_Tonnes"].sum()
avg_yield = filtered["Yield_Tonnes_per_Hectare"].mean()
avg_rainfall = filtered["Rainfall_mm"].mean()
total_area = filtered["Area_Hectares"].sum()
k1, k2, k3, k4 = st.columns(4)
k1.metric("Total production", f"{total_production:,.0f} t")
k2.metric("Average yield", f"{avg_yield:,.2f} t/ha")
k3.metric("Average rainfall", f"{avg_rainfall:,.0f} mm")
k4.metric("Total cultivated area", f"{total_area:,.0f} ha")

tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "Overview", "Crop Analysis", "Rainfall", "Regional Comparison", "Data & Insights"
])

with tab1:
    left, right = st.columns(2)
    yearly = filtered.groupby("Year", as_index=False).agg(
        Production_Tonnes=("Production_Tonnes", "sum"),
        Yield_Tonnes_per_Hectare=("Yield_Tonnes_per_Hectare", "mean"),
        Rainfall_mm=("Rainfall_mm", "mean"),
    )
    with left:
        fig = px.line(yearly, x="Year", y="Production_Tonnes", markers=True,
                      title="Total production by year", labels={"Production_Tonnes": "Production (tonnes)"})
        st.plotly_chart(fig, use_container_width=True)
    with right:
        crop_prod = filtered.groupby("Crop", as_index=False)["Production_Tonnes"].sum()
        fig = px.pie(crop_prod, names="Crop", values="Production_Tonnes", hole=0.35,
                     title="Production share by crop")
        st.plotly_chart(fig, use_container_width=True)
    fig = px.scatter(
        filtered, x="Rainfall_mm", y="Yield_Tonnes_per_Hectare",
        color="Crop", size="Area_Hectares", hover_data=["State", "Year", "Season"],
        title="Rainfall vs yield (bubble size = cultivated area)",
        labels={"Rainfall_mm": "Rainfall (mm)", "Yield_Tonnes_per_Hectare": "Yield (tonnes/ha)"}
    )
    st.plotly_chart(fig, use_container_width=True)

with tab2:
    crop_summary = filtered.groupby("Crop", as_index=False).agg(
        Average_Yield=("Yield_Tonnes_per_Hectare", "mean"),
        Production_Tonnes=("Production_Tonnes", "sum"),
        Area_Hectares=("Area_Hectares", "sum"),
    ).sort_values("Average_Yield", ascending=False)
    left, right = st.columns(2)
    with left:
        fig = px.bar(crop_summary, x="Crop", y="Average_Yield", color="Crop",
                     title="Average yield by crop", labels={"Average_Yield": "Yield (tonnes/ha)"})
        st.plotly_chart(fig, use_container_width=True)
    with right:
        fig = px.bar(crop_summary.sort_values("Production_Tonnes"), x="Production_Tonnes", y="Crop",
                     orientation="h", title="Total production by crop",
                     labels={"Production_Tonnes": "Production (tonnes)"})
        st.plotly_chart(fig, use_container_width=True)
    st.dataframe(crop_summary, use_container_width=True, hide_index=True)

with tab3:
    weather = filtered.groupby("Year", as_index=False).agg(
        Rainfall_mm=("Rainfall_mm", "mean"),
        Yield_Tonnes_per_Hectare=("Yield_Tonnes_per_Hectare", "mean"),
        Temperature_C=("Temperature_C", "mean"),
    )
```

```
        fig = px.line(weather, x="Year", y=["Rainfall_mm", "Temperature_C"],
                      markers=True, title="Average rainfall and temperature trend")
        st.plotly_chart(fig, use_container_width=True)
        fig = px.scatter(filtered, x="Rainfall_mm", y="Yield_Tonnes_per_Hectare",
                         trendline="ols", color="Season", hover_data=["State", "Crop", "Year"],
                         title="Rainfall-yield relationship")
        st.plotly_chart(fig, use_container_width=True)
        st.caption("A visible relationship is not proof that rainfall alone causes changes in yield.")

    with tab4:
        state_summary = filtered.groupby("State", as_index=False).agg(
            Average_Yield=("Yield_Tonnes_per_Hectare", "mean"),
            Production_Tonnes=("Production_Tonnes", "sum"),
            Average_Rainfall_mm=("Rainfall_mm", "mean"),
        ).sort_values("Average_Yield", ascending=False)
        left, right = st.columns(2)
        with left:
            fig = px.bar(state_summary.head(10).sort_values("Average_Yield"),
                         x="Average_Yield", y="State", orientation="h", color="Average_Yield",
                         title="Top 10 states by average yield",
                         labels={"Average_Yield": "Yield (tonnes/ha)"})
            st.plotly_chart(fig, use_container_width=True)
        with right:
            fig = px.bar(state_summary.sort_values("Production_Tonnes", ascending=False).head(10),
                         x="State", y="Production_Tonnes", color="State",
                         title="Top 10 states by total production",
                         labels={"Production_Tonnes": "Production (tonnes)"})
            st.plotly_chart(fig, use_container_width=True)
        st.dataframe(state_summary, use_container_width=True, hide_index=True)

    with tab5:
        st.subheader("Correlation heatmap")
        corr_cols = ["Rainfall_mm", "Temperature_C", "Humidity_pct",
                     "Area_Hectares", "Yield_Tonnes_per_Hectare", "Production_Tonnes"]
        corr = filtered[corr_cols].corr(numeric_only=True)
        fig = px.imshow(corr, text_auto=".2f", aspect="auto", color_continuous_scale="RdYlGn",
                        zmin=-1, zmax=1, title="Correlation between numeric variables")
        st.plotly_chart(fig, use_container_width=True)
        st.subheader("Filtered data")
        st.dataframe(filtered, use_container_width=True, hide_index=True)
        st.download_button(
            "Download filtered data (CSV)",
            data=filtered.to_csv(index=False).encode("utf-8"),
            file_name="filtered_crop_yield_rainfall.csv",
            mime="text/csv",
        )
        st.subheader("Quick insights")
        best_crop = filtered.groupby("Crop")["Yield_Tonnes_per_Hectare"].mean().idxmax()
        best_state = filtered.groupby("State")["Yield_Tonnes_per_Hectare"].mean().idxmax()
        st.write(f"- Highest average yield in the current selection: **{best_crop}**.")
        st.write(f"- Highest average-yield state in the current selection: **{best_state}**.")
        st.write(f"- Average rainfall in the current selection: **{filtered['Rainfall_mm'].mean():.1f} mm**.")
        st.write("- These insights describe the selected dataset only; the included demo data is simulated.")

    st.divider()
    st.caption("Built with Streamlit, Pandas, and Plotly | Crop Yield & Rainfall Intelligence Dashboard")
```

## B. Dataset Notes

- File: crop_yield_rainfall_dataset.csv, 600 rows, 10 columns, 40.4 KB, no missing cells.
- Coverage: years 2015–2024, 10 state/region labels, 6 crops (Sugarcane, Rice, Wheat, Maize, Groundnut, Cotton), 3 seasons (Kharif, Rabi, Summer).
- Columns: Year, State, Crop, Season, Rainfall_mm, Temperature_C, Humidity_pct, Area_Hectares, Yield_Tonnes_per_Hectare, Production_Tonnes.
- All measurements are simulated for educational demonstration and are not official or verified agricultural statistics.
- A replacement CSV must keep the same column names to work with the dashboard.

## C. Additional Screenshots

The Regional Comparison and Data & Insights tabs can be run with the command in Section 5.2; screenshots of these tabs may be added here if required by the department.